Functions in Python (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p3-functions)  (https://github.com/learn-co-curriculum/python-p3-functions/issues/new)

Learning Goals

- Understand the similarities between functions in JavaScript and Python.
- Identify key differences between functions in JavaScript and Python.
- Define functions with parameters.
- Call functions and use their return value.

Key Vocab

- **Interpreter**: a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell**: an interactive interpreter that can be accessed from the command line.
- **Data Type**: a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception**: a type of error that can be predicted and handled without causing a program to crash.
- **Code Block**: a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function**: a named code block that performs a sequence of actions when it is called.
- **Scope**: the area in your program where a specific variable can be called.

Introduction

One of the first things you likely learned in JavaScript was how to write functions. In this lesson, you'll get practice writing functions in Python to see the difference between Python functions and JavaScript functions.

Python Function Syntax

To start, let's try re-writing this JavaScript function in Python:

```
function myFunction(param) {
  console.log("Running myFunction");
  return param + 1;
}
```

As a quick recap of the syntax here:

- The `function` **keyword** identifies this code as a function.
- `myFunction` is a **variable name** we can use to refer to the function from elsewhere in our code, written in camel case by convention.
- The parentheses `()` after the function name give a space where we can define **parameters** for our function.
- `param` is the variable name given to our function's parameter; it will be assigned a value when the function is invoked and passed an argument.
- To define the body of the function, we use curly brackets (`{ }`).
- `console.log` is a method that will output information to the terminal; remember, this is *different* from a function's **return value**.
- The `return` keyword is needed when we want our function to return a value when it is called; in this case, it will return a value of whatever the `param` variable is plus one.

To actually run the code inside the function, we must invoke it:

```
const myFunctionReturnValue = myFunction(1);
// => "Running myFunction"
console.log(myFunctionReturnValue);
// => 2
```

Here, we're calling the function `myFunction` with an **argument** of `1`. We are then assigning the **return value** of `myFunction` to a new variable, `myFunctionReturnValue`.

If we wanted to write a method in Python with similar functionality, here's how it would look:

```
def my_function(param):
    print("Running my_function")
    return param + 1

# New code goes here!
```

There are a few key differences in the syntax here:

- Use the `def` keyword to identify this code as a function.
- Write the name of the method in snake case (by convention).
- Parameters are still defined in parentheses, after the method name.
- Instead of curly brackets, begin with a colon after the parentheses.
- In Python, we must indent all code that is meant to be executed in my_function. The **PEP-8 standards (https://peps.python.org/pep-0008/)** for writing Python code state that each indentation should be composed of four spaces (though the interpreter is less picky).
- `return` statements in Python work very similarly to those in JavaScript, but no semicolon is needed after the return value.
- Rather than closing with a curly bracket, any new code can be written at the original indentation level.

Run the Python shell and copy/paste the function definition above into your session. Then, run the function:

```
my_function_return_value = my_function(1)
# => Running my_function
# => 2
my_function_return_value
# => 2
```

When the `my_function()` function is called, you'll see the output from the `print()` function in the terminal, followed by the return value. The return value, `2`, is then saved to the variable `my_function_return_value`.

> You might see some functions referred to as **methods**. Methods are a special type of function that belong to **objects**. They often act upon those objects when called- remember `list.sort()` and `dict.get()` ?

Arguments

JavaScript allows you to define functions that expect a certain number of arguments, but will still run your code even if you don't pass in the expected number when you invoke the function. This can lead to some unexpected behavior in your JavaScript applications.

Consider the following:

```
function sayHi(name) {
  console.log(`Hi there, ${name}!`);
}

sayHi();
```

What do you think will happen when this code runs? Will it throw an error? Print something to the console? If so, what? Try running it in the browser to find out.

Unfortunately for JavaScript developers, bugs like these are hard to identify because they can only be found by testing our code and looking for unexpected behavior.

In Python, thankfully, when we run a method without passing in the required arguments it will give us an error message:

```
def say_hi(name):
    print(f"Hi there, {name}!")

say_hi()
# => TypeError: say_hi() missing 1 required positional argument: 'name'
```

Error messages like this are a **good thing** for us as developers, because it ensures that we are using methods as they are intending to be used, rather than trying to "fail gracefully" like JavaScript does.

Note that this mistake resulted in a `TypeError`. What *type* of input did we provide? What *type* did `say_hi()` expect?

Default Arguments

We can fix the behavior of our JavaScript function above by providing a default argument: a value that will be used if we don't explicitly provide one.

```
function sayHi(name = "friend") {
  console.log(`Hi there, ${name}!`);
}

sayHi();
// => "Hi there, friend!"
sayHi("Sunny");
// => "Hi there, Sunny!"
```

Python also lets us provide default arguments:

```
def say_hi(name="Engineer"):
    print(f"Hi there, {name}!")

say_hi()
# => "Hi there, Engineer!"

say_hi("Sunny")
# => "Hi there, Sunny!"
```

Return Values

You can categorize all functions that you write as generally useful for one (or both) of these things:

- What *return value* they have.
- What *side effects* they have (what other parts of the application they change; or what they output to the terminal; or what they write to a file; etc).

Writing output to the terminal using `console.log` or `print()` is a *side effect* of a function: it's distinct from the function's *return value*.

Consider these two JavaScript functions:

```
function addAndLog(num1, num2) {
  console.log(num1 + num2);
}

function addAndReturn(num1, num2) {
  return num1 + num2;
}

const sum1 = addAndLog(2, 2);
const sum2 = addAndReturn(2, 2);
```

What do you expect the values of `sum1` and `sum2` to be? What output would you expect to see in the console if you ran this code?

Since `addAndLog` doesn't use the `return` keyword, the value of `sum1` is undefined. We're only using `addAndLog` for its *side effect* of logging output to the terminal. `sum2`, on the other hand, will have a value of `4`, since we are using `addAndReturn` for its return value.

Think of it this way: methods are like vending machines. When you use a vending machine you just put in two arguments, the number (C7) and your money. We already know how to use arguments, but then your vending machine might do two things. One, it will make a noise saying that everything worked, beep beep. Then it gives you the soda. The soda is the return type. But those beeps? Are you able to do anything with them? Nope! That's like `print()`: it just tells you stuff and then goes into the ether! Gone forever.

Like in JavaScript, you must use the `return` keyword to retrieve an output value from a function in Python.

Let's take a look:

```
def stylish_painter():
    best_hairstyle = "Bob Ross"
    return "Jean-Michel Basquiat"
    return best_hairstyle
    print(best_hairstyle)

stylish_painter()
```

What do you expect the return value of the above method to be? Go into the Python shell, copy and paste the above method and call it.

You may have expected the return value to be Bob Ross. His name is the last line of the function. *However*, the return value of the above method is actually Jean-Michel Basquiat! The `return` keyword will disrupt the execution of your function, and prevent Python from running any lines of code after the `return` keyword.

---

pass

There will be times when you're writing out your code and know that you will need a function later, but you don't quite know what to put in there yet. A good practice in Python development is to make use of the `pass` keyword in empty functions until they are ready to be fleshed out.

```
def my_future_function():
    pass
```

Because Python uses indentation to determine when a code block starts and ends, it is necessary to put *something* inside of an empty function- comments, sadly, do not count.

Python developers typically opt for `pass` over `return None` because it is a statement rather than an expression. It does not terminate the function like a `return` statement would do. You can even put code after your `pass` and it will be executed! A `pass` statement reminds you

that there is work to be done without interfering with your development.

Instructions

In the `js/index.js` file, there are four functions defined in JavaScript. Your job is to recreate the functionality of those functions by writing methods in Python that will accomplish the same thing.

Write your code in `lib/functions.py`. Run `pytest -x`, and use the tests along with the code in `js/index.js` to guide your work.

1. Define a method `greet_programmer()` that takes no arguments. It should output the string "Hello, programmer!" to the terminal with `print()`.
2. Define a method `greet()` that takes one argument, a name. It should output the string "Hello, name!" (with "name" being whatever value was passed as an argument) to the terminal with `print()`.
3. Define a method `greet_with_default()` that takes one argument, a name. It should output the string "Hello, name!" (with "name" being whatever value was passed as an argument) to the terminal with `print()`. If no arguments are passed in, it should output the string "Hello, programmer!".
4. Define a method `add()` that takes two numbers as arguments and **returns** the sum of those two numbers.
5. Define a method `halve()` that takes one number as an argument and **returns** the that number's value, divided by two.

Conclusion

Python's function syntax has a few things that make them distinct from JavaScript functions. In particular, make sure you pay attention to the **indentation levels** of the code inside of Python functions, and always call functions with the right number of arguments to avoid errors.

Resources

- **Defining Your Own Python Function (https://realpython.com/defining-your-own-python-function/)**

This tool needs to be loaded in a new browser window

Load Functions in Python (CodeGrade) in a new window